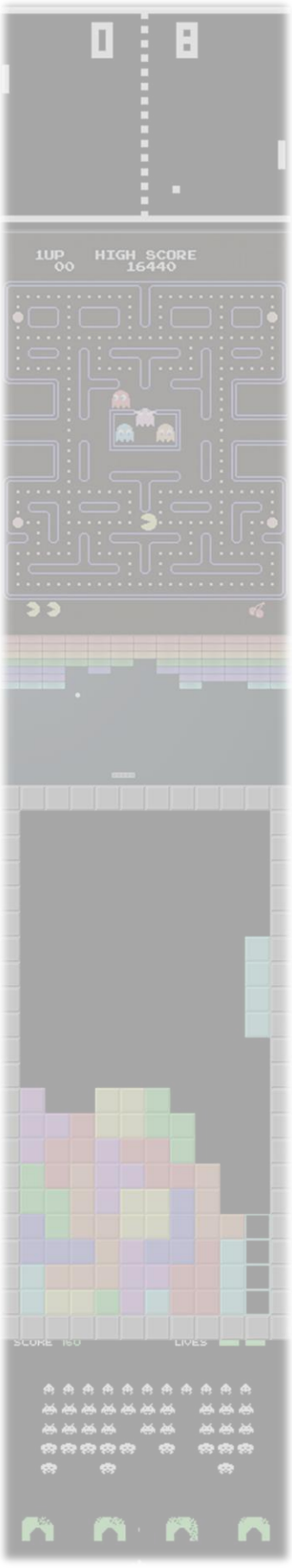
FROM CODE TO GAME

BEGINNING 2D GAME PROGRAMMING WITH SDL3



TIMELINE

- 10 November 2026
  - Online training: Using the basic functionalities of the SDL3 library
- 14 November 2026
  - CodeRun @TUCN

TUCN TRAINER

- Adrian SABOU – Senior Lecturer, Computer Graphics and Interactive Systems (CGIS) Group, Department of Computer Science, Faculty of Automation and Computer Science, Technical University of Cluj-Napoca (TUCN)

TECHNICAL SPECIFICATIONS

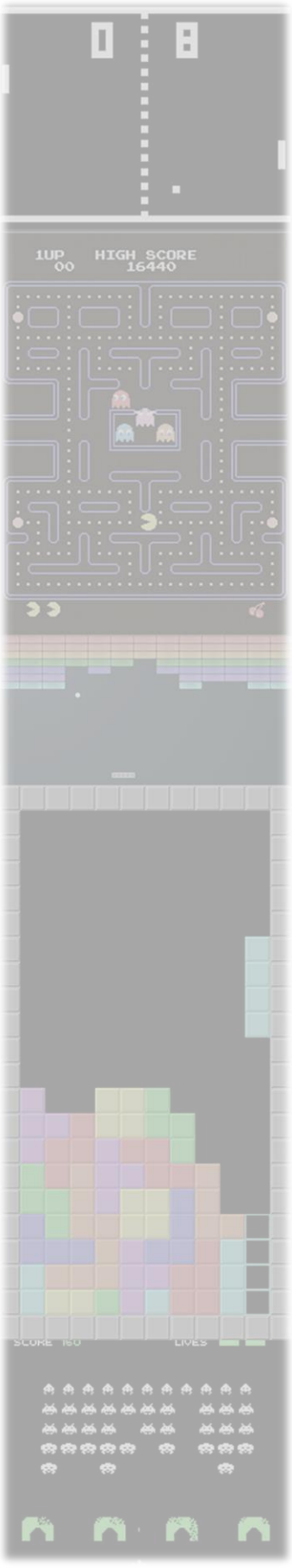
- Required operating system: any operating system for which the SDL3 library has support (Microsoft Windows, Linux, macOS)
- Programming languages: C/C++
- Required IDEs: any C/C++ capable IDE (e.g. Microsoft Visual Studio, CLion, Xcode, etc.)
- Auxiliary libraries: The SDL3 library for programming 2D Games (https://www.libsdl.org), The OpenGL mathematics library (GLM) (https://glm.g-truc.net/0.9.9/)
- Prerequisites: Basic C/C++ programming skills
- Pre-work: Configuring your IDEs to compile and run a "Hello SDL3 Window" Application (link to code). You can use any tutorials available online for this task. **The code provided must compile as it is, without any alterations to it!** Pre-configured projects are available for Microsoft Visual Studio 2022 (Microsoft Windows), CLion 2024.2.2 (Microsoft Windows, both for the VS2022 and the bundled MinGW Toolchains), CLion 2024.2.2 (macOS) (link to projects). For CLion on macOS, please consult the following document for the instructions to follow after you download the project (link to document).

TRAINING OUTLINE

- Trainer introduction
- SDL3 library overview
- Understanding a basic SDL3 application (Hello SDL3 Window!)
- Basic user interaction in SDL3
- Drawing basic shapes in SDL3
- Working with time in SDL3
- Putting it all together in a simple mini game

A Blast From the Past

Implementing a Vertical Scrolling Shooter Game



Context

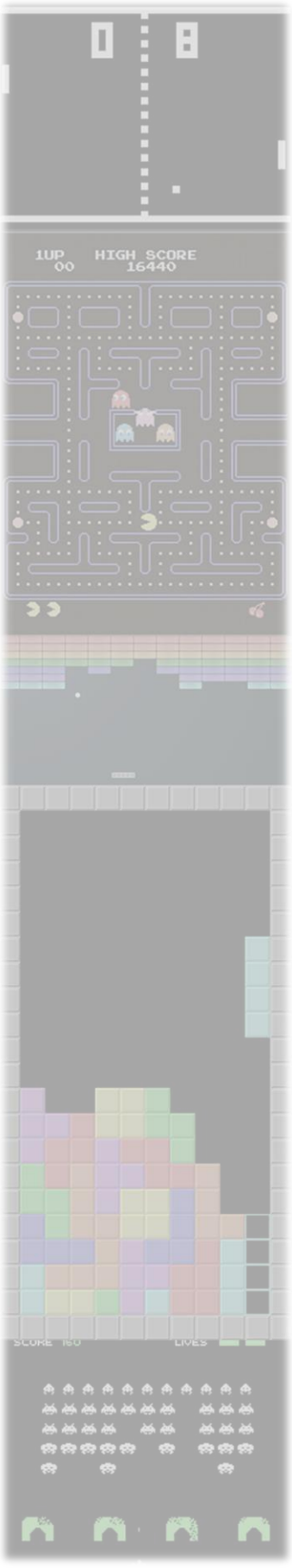
- In 1982, Namco released the world's first vertical scrolling shooter game, Xevious. Gameplay involved the player controlling a ship flying over a landscape using a top-down view, while shooting at air targets and bombing ground targets. The vertical scrolling shooter landscape continued to evolve with several other games, Jaleco's Exerion being the first to implement an inertial ship movement system, culminating in 1999 with Epic MegaGames's Tyrian 2000.



- In this context, your task is to implement a very simple vertical scrolling shooter game in SDL3.
- You will start from the from the SDL3 application **template** provided here. **Using the SDL3 library and the code template is mandatory and implementations that do not use them will not be considered for scoring.**

Easy Task

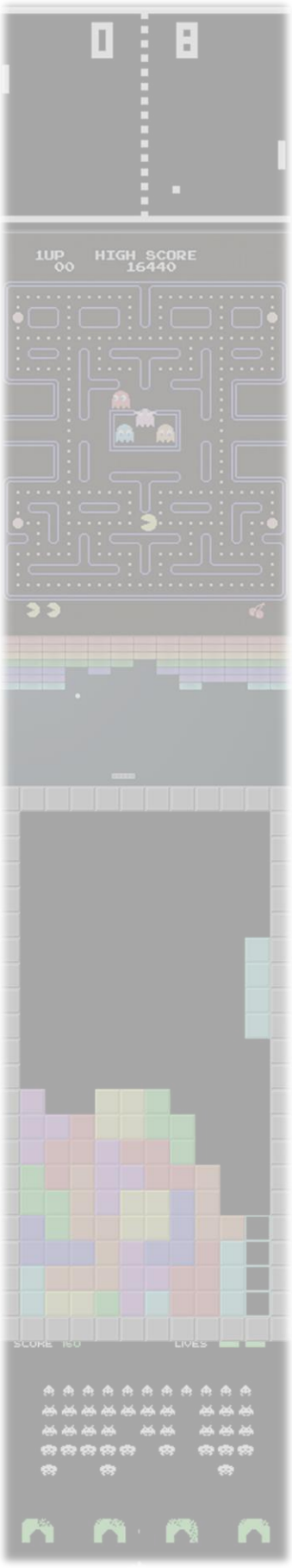
- Your game will be similar to the following demo (demo link)
- The goal of the game is to destroy all enemies on screen while avoiding being destroyed by them
- Your SDL3 window will be 700 pixels wide and 800 pixels tall
- The vertical scrolling landscape is already provided in the **template**
- The player's ship will be a green rectangle, 50 pixels wide and 50 pixels tall, initially positioned on the central line of the window, at a distance of half its height from the window's lower edge
- At the start of the game, 3 enemies will appear in the scene, according to the following rules:
  - Enemies will be orange rectangles
  - Enemies will have the same size as the player's ship
  - Vertically, each enemy will be positioned randomly in the upper half of the window
  - Horizontally, each enemy will be positioned randomly in its own vertical slice (e.g. for 3 enemies, split the window into three equal horizontal slices and place one enemy, randomly, in each slice)
- The player can use the mouse to move the ship inside the window (without allowing the ship to exit the window), the ship accurately following the mouse; the mouse cursor should not be visible, and the window should capture mouse movement while the game is running (tip: see *SDL_SetWindowRelativeMouseMode*)
- The player can press the left mouse button to begin firing projectiles upon the enemies



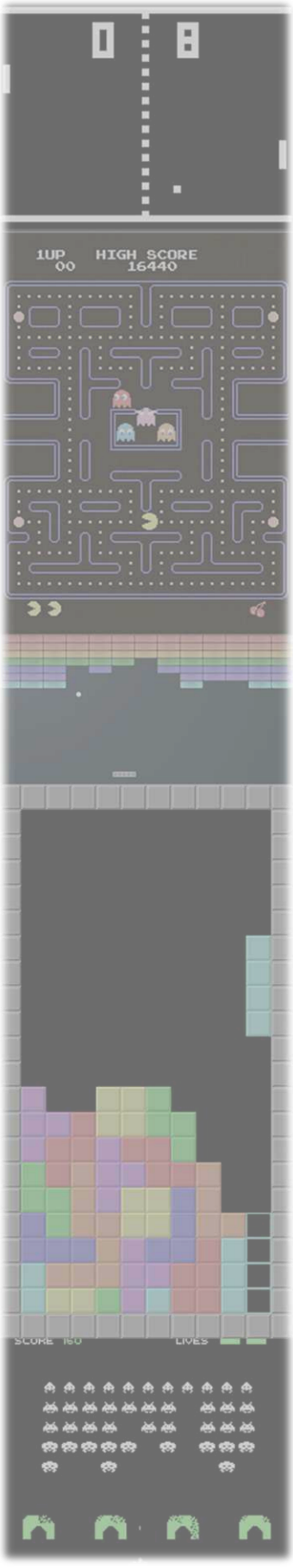
  - Projectiles are red circles having a 10 pixels radius, initially positioned in the center of the player's ship, and constantly travelling upwards with a velocity of 500 pixels per second
  - While the left mouse button is pressed, projectiles will be continuously generated at a rate of one projectile each 0.3 seconds
  - When the left mouse button is released, projectiles stop being generated
- After 1 second of gameplay, enemies will start firing upon the player's ship
  - Enemy projectiles are magenta circles having a 10 pixels radius, initially positioned in the center of enemy ships, and constantly travelling towards the player ship's position at the moment of firing, with a velocity 5 times smaller than that of player projectiles
  - Active (not destroyed) enemies continue to fire upon the player's ship with a firing rate 10 times lower than player projectiles
- When a player projectile collides with an enemy ship, the enemy and the projectile are destroyed
- When the player's ship collides with an enemy ship or enemy projectile, the game ends with a loss for the player
- When all enemies have been destroyed, the game ends with victory for the player
- The moment the game ends, all movement (including vertical scrolling and ship) stops, all enemies and projectiles disappear and, if the user lost, the ship will turn red, while if the user won, the ship will turn blue
- After the game ends, it can be reset to its original state using the R key
- The game cannot bet reset before the game ends
- **Important notes!**
  - **All placements are relative to their center**
  - **All positional constraints must be fulfilled even if the size of the window changes (so no hard coding positions based on the 700x800 window size)**
  - **You must design your game to allow it to run indefinitely**

HARD TASK

- Your game will be similar to the following demo (demo link)
- The goal of the game is to destroy all enemies on screen while avoiding being destroyed by them
- Your SDL3 window will be 700 pixels wide and 800 pixels tall
- The vertical scrolling landscape is already provided in the **template**
- The player's ship will be a green rectangle, 50 pixels wide and 50 pixels tall, initially positioned on the central line of the window, at a distance of half its height from the window's lower edge
- At the start of the game, enemies will appear in the scene, according to the following rules:
  - Enemies will be orange rectangles
  - Enemies will have the same size as the player's ship
  - Enemies will enter the window in the upper part and move downwards with a constant velocity randomly chosen between 50 and 150 pixels per second



    - Horizontally, each enemy will be positioned randomly in its own vertical slice (e.g. for 3 enemies, split the window into three equal horizontal slices and place one enemy, randomly, in each slice)
    - There can only be one active enemy in each horizontal slice at once
    - Every 3 seconds, if a slice is clear of its enemy an additional enemy is generated for it
- The player can use the mouse to move the ship inside the window (without allowing the ship to exit the window), following an inertial movement system as described below; the mouse cursor should not be visible, and the window should capture mouse movement while the game is running (tip: see *SDL_SetWindowRelativeMouseMode*)
    - The ship will accelerate when the mouse is moved based on the amount of its relative movement (tip: see currentEvent.motion.xrel and currentEvent.motion.yrel) and a strength of 400 pixels per squared seconds up to a maximum velocity of 500 pixels per second
    - When the mouse stops moving, the ship decelerates with a strength 2 times that of its acceleration
- The player can press the left mouse button to begin firing projectiles upon the enemies
    - Projectiles are red circles having a 10 pixels radius, initially positioned in the center of the player's ship, and constantly travelling upwards with a velocity of 500 pixels per second
    - While the left mouse button is pressed, projectiles will be continuously generated at a rate of one projectile each 0.3 seconds
    - When the left mouse button is released, projectiles stop being generated
- Enemies will start firing upon the player's ship 1 second after they enter the window
    - Enemy projectiles are magenta circles having a 10 pixels radius, initially positioned in the center of enemy ships, and constantly travelling towards the player ship's position at the moment of firing, with a velocity 5 times smaller than that of player projectiles
    - Each enemy will fire an additional 2 projectiles alongside the main one, the directions of the additional projectiles being rotated with 15° and -15° from the main projectile's direction
    - Active (not destroyed) enemies continue to fire upon the player's ship with a firing rate 10 times lower than player projectiles
- To survive the extra projectiles, the player can use its special weapon, the "Repulsor", which, when the right mouse button is pressed, slows down all enemy projectiles with 10 pixels per second by pushing them away from the player's ship; this effect can only happen once per right mouse press, but repeated presses can be used undefinedly, even to the point where projectiles start moving away from the player's ship
- When a player projectile collides with an enemy ship, the enemy and the projectile are destroyed
- When the player's ship collides with an enemy ship or enemy projectile, the game ends with a loss for the player
- When all enemies have been destroyed, the game ends with victory for the player



- The moment the game ends, all movement (including vertical scrolling and ship) stops, all enemies and projectiles disappear and, if the user lost, the ship will turn red, while if the user won, the ship will turn blue
- After the game ends, it can be reset to its original state using the R key
- The game cannot bet reset before the game ends
- **Important notes!**
  - **All placements are relative to their center**
  - **All positional constraints must be fulfilled even if the size of the window changes (so no hard coding positions based on the 700x800 window size)**
  - **You must design your game to allow it to run indefinitely**

Easy Task Scoring

- Attendance: 10 points
- Correct visual implementation of all game elements (ship, enemies, projectiles): 10 points
- Correct implementation of the ship's constrained movement: 10 points
- Correct implementation of ship projectiles and firing mechanism: 20 points
- Correct implementation of enemy projectiles and firing mechanism: 10 points
- Correct detection and solving of all collisions (between projectiles and ships and between ships): 20 points
- Correct implementation of game winning, losing and resetting scenarios: 10 points
- Constant, fluid and correct movement speed of all game objects: 10 points
- Time bonus: maximum 10 points (0.5 points / minute, for a maximum of 20 minutes); **Note:** to receive the time bonus, the task needs to be at least 50% solved correctly

Hard Task Scoring

- Attendance: 20 points
- Correct visual implementation of all game elements (ship, enemies, projectiles): 20 points
- Correct implementation of the ship's inertial constrained movement: 40 points
- Correct implementation of ship projectiles and firing mechanism: 20 points
- Correct implementation of enemy projectiles and firing mechanism: 30 points
- Correct implementation of the "Repulsor": 30 points
- Correct detection and solving of all collisions (between projectiles and ships and between ships): 20 points
- Correct implementation of game winning, losing and resetting scenarios: 10 points
- Constant, fluid and correct movement speed of all game objects: 10 points
- Time bonus: maximum 20 points (1 point / minute, for a maximum of 20 minutes); **Note:** to receive the time bonus, the task needs to be at least 50% solved correctly